Hybris

Coding Standards Part - 2

Document History

Version	Reason for change	Date	Modified by	Reviewed By
1.0	Initial Draft	September 13, 2017	Anurag Mundra	Srinivas Jayanna
1.1	Formatting changes	November 14, 2017	Anurag Mundra	Srinivas Jayanna

Colour Legend

	Reason for change	Reason	Created/Modified By
1	Blue background	Warning	Anurag Mundra
2	Yellow background	Information	Anurag Mundra

Table of Contents

1 Responsive Themes 5
1.1 Theme Files Structure 5
1.1.1 View Files 5
1.1.2 Theme resources 5
1.1.3 Generated Variables 6
1.2 Build Tools 7
1.2.1 Using the Grunt Build Process 7
2 Development Environment 8
2.1 Sample Project Setup 8
2.2 Custom LESS Files Structure 8
2.2.1 Sample LESS Files 10
2.3 Responsive Storefront Build Process Enhancements 10
2.3.1 Further Enhancements 11
3 LESS and CSS Tools and Utilities 15
3.1 IDE Support 15
3.1.1 Eclipse 15
3.1.2 IntelliJ 15
3.2 Browser Tools 16
3.2.1 Firefox 16
3.2.2 Chrome 16
4 Additional Development Tools 17
4.1 Sample Configuration 17
4.1.1 How to Use 17
4.2 Additional Grunt Tasks for Build Process 18
4.2.1 Using project properties in Grunt build process 18
4.2.2 LESS/CSS Postprocessing 18
4.3 Helpers and Utilities 20
4.3.1 Code Quality Reports 20
4.3.2 Image Minification 22
4.3.3 Webfont Generation 24
5 Usage Guidelines 24
Before you Start 24
5.1 General Rules 25
5.1.1 Write DRY Code 25
5.1.2 Keep the Specificity Low 25

5.2 Project Structure 25
5.2.1 Modules 25
5.2.2 Theme Variables 26
5.2.3 Theme Mixins 26
5.2.4 Theme Components 26
5.2.5 Theme Elements 27
5.2.6 Temporary Adjustments 27
5.2.7 Main theme stylesheet 27
5.3 Naming Conventions 27
5.4 Code Formatting 28
5.5 Nesting Selectors 29
5.6 Data Types 30
5.6.1 Numbers and Units 30
5.6.2 URLs 30
5.6.3 Colors 30
5.7 Mixins 31
5.8 Media Queries 31
5.9 Vendor Prefixes 32
5.10 Comments 32
6 References 33
6.1 Links 33

In addition to responsive templates for pages and CMS components, which enable adjusting layouts to the device size, the responsive storefronts also provide support for LESS - the CSS preprocessor. This makes layout customization much more efficient, ensures visual consistency and helps to improve network and browser performance.

This section describes workflow and best practices for working with responsive storefront themes.

1 Responsive Themes

The standard workflow for custom responsive themes is as follows:

1. Create custom theme (e.g. by copying one of the existing ones; since 6.1 there is dedicated Ant target facilitating this task).
2. Add required resources (fonts, images etc.), modify existing LESS files and add custom ones.
3. Build project with one of build tools described below. Source files will be compiled and saved in the destination folder.
4. Repeat steps 2 and 3 whenever needed.

When working with responsive themes, compiled and synchronized files should not be edited directly. All changes should be made in source files.

Since version 6.0, responsive theme compilation is included in Ant build target by default. This means that all changes made to compiled CSS files, copied resources etc. **will be overwritten** during the build.

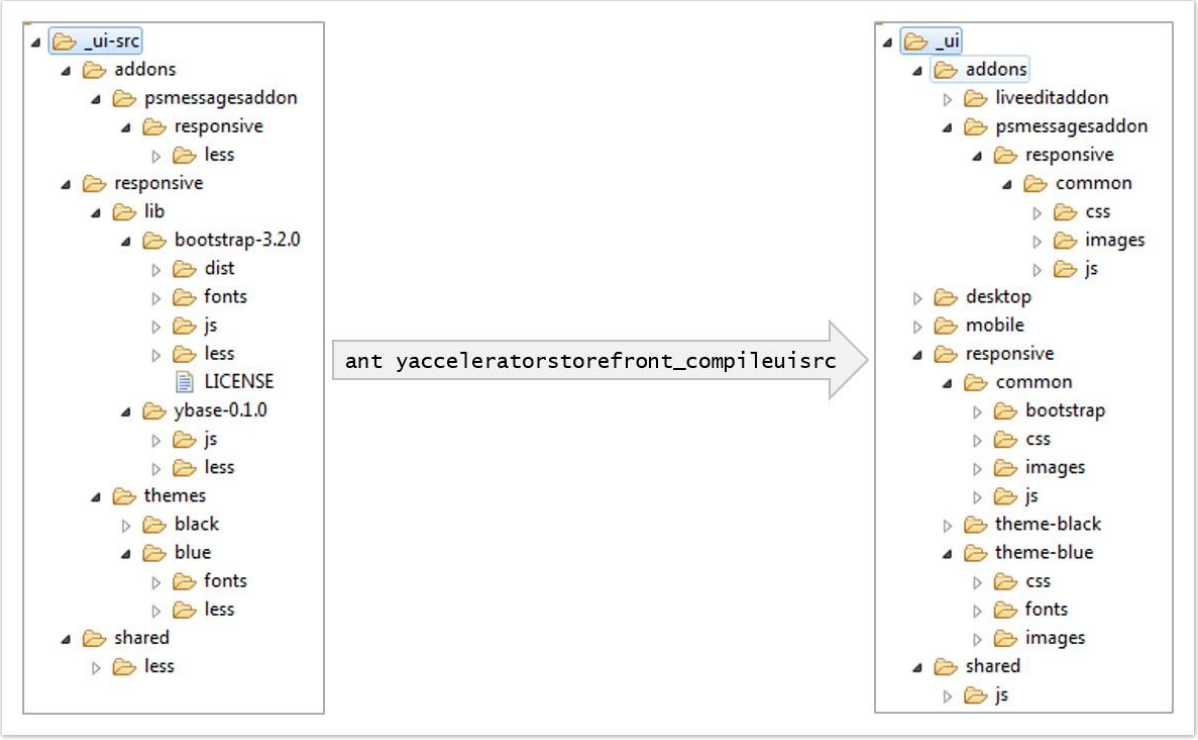
1.1 Theme Files Structure

1.1.1 View Files

JSP views and tags for the responsive experience are located in <extension folder>/web/webroot/WEB-INF/view/responsive and <extension folder>/web/webroot/WEB-INF/tags/responsive respectively. Note that not all WCMS components are available for the responsive experience and some of components have variants dedicated for the responsive storefront specifically. See Responsive Storefronts (hybris Help) for details.

1.1.2 Theme resources

The theme's source files (LESS files, fonts, JS libraries etc.) are located in <extension folder>/web/webroot/WEB-INF/_ui-src. The shared subfolder contains common settings; theme-specific sources are located in respective subfolders under the responsive folder. They are synchronized and compiled to corresponding locations within the web/webroot/_ui directory.



Responsive theme - source and destination folder structure

See Responsive Website Build Process for details on build process.

While the fonts, JS libraries and stylesheets are based on the source files, there are still some static resources that in default setup are placed in the destination web/webroot/_ui directory. See Responsive storefront build process enhancements for recommendations on more uniform approach.

Responsive themes are based on global styles, defined by the *ybase* library. This library, in turn, is based on Bootstrap framework and extends it with its own set of variables, mixins and style definitions.

High-level LESS dependency overview in default responsive theme structure

Bootstrap library version 3.2.0 is used.

1.1.3 Generated Variables

Generated variables allow sharing settings between JSP, JS and CSS files, as described in JS, LESS and JSPs: Build Process for Generating Variables. They are set based on contents of resources/generateVariables.properties file. Resulting files (generatedVariables.tag, generatedVariables.js, and generatedVariables.less) should not be manually modified.

Since the variable names are shared between JS, LESS and JSP, they cannot contain a "-" character, used in Bootstrap naming convention. To enable setting those variables via generated properties, the variableMapping.less file is provided.

By default, the resulting generatedVariables.* files are placed in shared directory. This needs to be taken into consideration when developing multiple themes in the same project.

1.2 Build Tools

There are two build tools provided, described in Responsive Website Build Process: Ant and Grunt. Both of these tools perform the same set of operations: synchronize specified resources and compile LESS source files into final CSS, although the Grunt does not compile LESS files from addons. With that exception, they both produce equivalent results.

The Ant build is performed by default at the end of the storefront build process via the <storefront_extension>_compileuisrc Ant target. It may be also triggered manually via the ant <storefront_extension>_compileuisrc command.

The configuration for Grunt build process is provided in the Gruntfile.js file. It requires installing Node.js., Grunt and Grunt CLI.

Sometimes the project requirements make it impossible to provide proper configuration to use the Grunt build process in production environment. In those cases even if the Grunt tools are used during the development, the code should always be built with Ant target and verified before publishing changes.

During the development phase, using the Grunt seems to have two advantages over the Ant target. First, it is usually noticeably faster, which is very convenient given the nature of frontend development process, where minor changes are frequently introduced and should be instantly visible in the browser. Second, the watch task is configured out of the box, eliminating the necessity for a manual compilation step.

Nonetheless, in implementation projects requiring extensive theme modifications, none of the default configurations proved to fully satisfy the needs for storefront customization. See the Sample project setup section for ideas on possible improvements.

1.2.1 Using the Grunt Build Process

Using the Grunt process requires installing Node.js and Grunt CLI.

In OSX you may need to use sudo for installing Grunt, Grunt CLI and plugins. See http://gruntjs.com/getting-started for details.

1. Install Node.js. Add Node.js directory to path, if it hasn't been updated automatically.
2. Install Grunt CLI:

   ```
   > npm install -g grunt-cli
   ```

3. Install Grunt:

   ```
   > npm install -g grunt
   ```

4. Open console in the storefront extension folder (where package.json and Gruntfile.js are located).
5. Install project's dependencies (appropriate plugins will be determined based on package.json configuration):

```
> npm install
```

2 Development Environment

2.1 Sample Project Setup

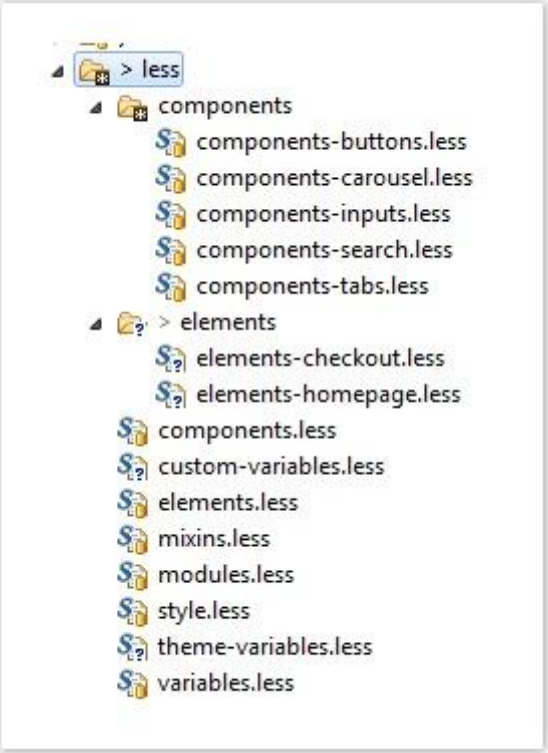
This section describes sample setup recommended for implementation project including theme customization. It should be adjusted to specific project requirements.

Depending on the level of customization needed, there are several options of theme configuration:

- In the simplest cases (e.g. when only color scheme or font faces need to be modified), overriding theme variables may be enough to achieve desired results.
- When more extensive modifications need to be introduced, it is recommended to create additional custom LESS files and import them into theme's style.less file. This clearly separates default theme's settings from the customization, which allows evaluating the scope of changes at a glance.
- In complex cases, when significant areas of site's look and feel are changed, the approach from the previous point may result in excessive overrides - the default styles are included only to be overwritten by the custom ones. In such cases it may be better to not import the whole ybase.less file into style.less, but pick only the relevant components and include them in the custom base file (kind of custom build pattern).
- Sometimes it may be necessary to just bypass standard styling and build the custom theme from scratch.

2.2 Custom LESS Files Structure

For readability and reusability sake, it is recommended to clearly separate custom modifications from the original theme code. Proper LESS file structure makes those changes easy to evaluate and extract.



Sample structure of custom theme's LESS files

The style.less, variables.less and theme-variables.less are generated when the initial theme is created. Then, variables.less and style.less are modified to import custom-variables.less and other custom files; all direct styling is placed in those custom files - the original ones only contain imports.

The table below explains the purpose of individual custom files. See the LESS usage guidelines section for detailed explanation.

File	**Purpose**
custom-variables.less	custom variables introduced by the theme and overrides for default Bootstrap and *ybase* variables; this file is optional, as all those variables can be created or modified in theme-variables.less, but having a custom, theme-specific file enables better separation of default and custom content.
modules.less	universal, project-agnostic modules (browser vendor prefix mixins, reusable positioning mixins etc.)
modules/modules-*.less	definitions of various module types, if it is necessary to split the modules.less file (e.g. for autoprefixing vendor mixins)
mixins.less	mapping of modules.less to a theme-specific look and feel, as well as any other theme-specific mixins necessary
components.less	styling of theme-specific components that are reused on different pages across the site

File	Purpose
components/components-*.less	styling for components of given types or groups, if there are too many components to directly include them in the components.less file
elements.less	styling of one-off elements that are not repeatedly used
elements/elements-*.less	styling of elements for pages or page sections if there are too many components to directly include them in the elements.less file
todo.css	temporary adjustments delivered by the external team as pure CSS code, to be backported into the LESS structure
style.css	main theme stylesheet

2.2.1 Sample LESS Files

Sample LESS files for the custom responsive theme have been attached to this page, as described in Project structure in the LESS usage guidelines section. The folder structure is based on standard Accelerator 6.0/6.1 setup. To use the sample files:

- Create the new responsive theme (in 6.0 version you will need to manually copy, modify and import the new theme).
- Unzip the less-structure.zip and copy the contents of less folder over the original theme's less folder. This would add custom less files and replace original style.less and variables.less to include required import directives.

Sample files contain some examples of module and mixin definitions. The unprefixed version of modules-vendors.less file was also provided - it can be processed by Autoprefixer or manually adjusted to the project's requirements.

The sample setup does not contain the theme-variables.less file - it is generated during theme creation.

2.3 Responsive Storefront Build Process Enhancements

Between 5.4 version, when responsive templates were first introduced, and the current version, many small improvements have been made to the Ant build process (fixing OS-related path issues, forcing LESS compilation when any *.less file is changed as opposed to the style.less only etc.).

Since SAP Hybris Commerce 6.0, responsive resources compilation is included in the Ant build process by default. This means that all changes made to compiled CSS files, copied resources etc. **will be overwritten** during the build.

Since SAP Hybris Commerce 6.1, there is new Ant target: <storefront_extension>_responsive_theme_setup, facilitating creation of new themes. Also in this version default themes have been renamed from *blue* and *black* to *alpha* and *lambda* respectively. See Creating New Themes in hybris Help for details.

2.3.1 Further Enhancements

During the implementation of several projects including responsive storefronts, a few steps for further build process optimizations were determined.

All responsive storefront-related build tasks for Ant and Grunt processes are defined in <storefrontextension>/buildcallbacks.xml and <storefrontextension>/Gruntfile.js files respectively.

For each of the enhancements recommended below, a code sample for either Ant or Grunt build process has been provided. For the consistency sake, similar changes would need to be made for the other build process.

Moving all Theme Resources to the Source Folder

Moving all theme resources (images, static CSS files, custom theme scripts etc.) to the respective subfolders of web/webroot/WEB-INF/_ui-src/themes/<theme_name> folder allows keeping all the resources in single location. Contents of the destination folder (web/webroot/ui) can then be treated as any compiled code (not tracked in version control systems etc.). This approach prevents the accidental overrides and eliminates potential confusion due to the resources being scattered between various folders.

This step requires modifying Ant and Grunt building tasks to include additional resources.
Example of adding custom theme JS and images to UI source compilation Expand source
expand source?

```
<macrodef name="storefrontextensionstorefront_movelibs">
   <sequential>
      <!-- ... -->
      <copy todir="${ext.storefrontextensionstorefront.path}/web/webroot/_ui/responsive/">
         <fileset dir="${ext.storefrontextensionstorefront.path}/web/webroot/WEB-INF/_ui-
src/responsive/" includes="themes/**/js/*"/>
         <regexpmapper from="themes/(.*)/js/(.*)" handledirsep="yes" to="theme-
\1/js/\2"/>
      </copy>
      <copy todir="${ext.storefrontextensionstorefront.path}/web/webroot/_ui/responsive/">
         <fileset dir="${ext.storefrontextensionstorefront.path}/web/webroot/WEB-INF/_ui-
src/responsive/" includes="themes/**/images/*"/>
         <regexpmapper from="themes/(.*)/images/(.*)" handledirsep="yes" to="theme-
\1/images/\2"/>
      </copy>
   </sequential>
</macrodef>
```

Adding the Clean Process for Theme Resources

If all theme resources are moved to the source folder, it makes sense to also create corresponding build tasks to clean compiled files before generating new versions. This ensures that when source files are removed or renamed, those changes are reflected in compiled ones (it is not an issue with the standard setup, as it includes only defined set of JS libraries and single style.less file, so the resulting files are simply overridden).

This step requires creating new Ant and Grunt tasks.

Sample definition of the UI resource cleaning task Expand source
expand source?

```
<macrodef name="storefrontextensionstorefront_cleancompileduidir">
   <sequential>
      <delete dir="${ext.storefrontextensionstorefront.path}/web/webroot/_ui/responsive/"
includes="theme-*/**"/>
   </sequential>
</macrodef>

<target description="clean the compiled resources in _ui"
name="storefrontextensionstorefront_cleancompiledui_executor">
   <storefrontextensionstorefront_cleancompileduidir />
</target>

<macrodef name="storefrontextensionstorefront_after_clean">
   <sequential>
      <!-- ... -->
      <storefrontextensionstorefront_cleancompiledui_executor />
   </sequential>
</macrodef>
```

Extending the Build Process with Additional Files

In the standard Accelerator setup, only the style.less file is compiled to CSS. It may be necessary to include additional LESS source files (e.g. when separate stylesheets need to be included by specific pages).

This step requires modifying Ant and Grunt building tasks to include additional resources.
Example of adding custom other.less file to the LESS compilation macro

```
<macrodef name="storefrontextensionstorefront_lesscss">
   <sequential>
      <!-- .. -->
      <apply dir="${ext.storefrontextensionstorefront.path}/web/webroot/WEB-INF/_ui-
src/responsive/themes/" executable="java" failonerror="true" force="true" parallel="false">
         <!-- Give the input bundle of less files-->
         <fileset dir="${ext.storefrontextensionstorefront.path}/web/webroot/WEB-INF/_ui-
src/responsive/themes/">
            <include name="**/style.less"/>
            <include name="**/other.less"/>
         </fileset>
         <!-- .. -->
      </apply>
   </sequential>
</macrodef>
```

Parametrizing Build Process with the Theme Name

In most cases, only a single custom theme is delivered for any given implemented storefront, but even then, the original theme is kept within the project as a reference during development. As the UI resources compilation for every existing theme adds (sometimes

significant) time to the build process, it may be necessary to trigger compilation for the custom theme only. In order to do that, UI build tasks can be parametrized with the theme name argument so only the specified theme is compiled during the project build.

This step requires modifying Ant and Grunt building tasks to specify parameterized resources path.

UI build tasks customized with theme name Expand source

expand source?

```
<!-- .properties file containing theme-related properties -->
<property file="${ext.storefrontextensionstorefront.path}/project.properties" />

<macrodef name="storefrontextensionstorefront_movelibs">
   <!-- add the `theme` attribute to macro definition -->
   <attribute name="theme" />
   <sequential>
      <!-- ... -->
      <copy todir="${ext.storefrontextensionstorefront.path}/web/webroot/_ui/responsive/">
         <!-- use the `theme` attribute to determine resources path -->
         <fileset dir="${ext.storefrontextensionstorefront.path}/web/webroot/WEB-INF/_ui-
src/responsive/" includes="themes/@{theme}/**/fonts/*"/>
         <regexpmapper from="themes/(.*)/fonts/(.*)" handledirsep="yes" to="theme-
\1/fonts/\2"/>
      </copy>
      <!-- ... -->
   </sequential>
</macrodef>

<macrodef name="storefrontextensionstorefront_compileuisrc_executor">
   <!-- add the `theme` attribute to macro definition -->
   <attribute name="theme" />
   <sequential>
      <!-- pass the `theme` parameter to the called macro -->
      <storefrontextensionstorefront_movelibs theme="${theme.name}"/>
      <!-- ... -->
   </sequential>
</macrodef>

<macrodef name="testfrontstorefront_after_build">
   <sequential>
      <!-- ... -->
      <!--
      By default, only the theme specified in
`storefront.themes.compileUiSrcDuringBuild.pattern` property will be compiled during
project build
      -->
      <testfrontstorefront_compileuisrc_executor
theme="${storefront.themes.compileUiSrcDuringBuild.pattern}" />
      <!-- ... -->
   </sequential>
</macrodef>
```

```
<target description="compile the _ui-src folder in _ui"
name="storefrontextensionstorefront_compileuisrc">
   <!--
      Add the `theme.name` argument to the build target with default value of `*`
      The task can then be called with parameter:
      ant storefrontextensionstorefront_compileuisrc -Dtheme.name=customTheme
   -->
   <property name="theme.name" value="*"  />
   <storefrontextensionstorefront_compileuisrc_executor theme="${theme.name}"/>
</target>
```

Adding the Automatic Reload Feature

During the UI development, it is convenient to have recompiled resources automatically reloaded in the browser, so all changes are immediately reflected without manually refreshing the page. It is particularly beneficial when styling parts of the UI that are dynamically displayed in reaction to the user's behaviour (e.g. popups requiring multiple manual steps to be shown). This can be easily achieved with the Grunt build process, using the watch task's livereload option.

Sample configuration of the automatic reload in Gruntfile.js

```
module.exports = function(grunt) {
  grunt.initConfig({
    pkg: grunt.file.readJSON('package.json'),
    watch: {
       //...
       livereload: {
          options: {
             livereload: {
                host: 'localhost',
                port: 35729,
                key: grunt.file.read('path/to/server.key'),
                cert: grunt.file.read('path/to/server.crt')
             }
          },
          files:
['web/webroot/_ui/responsive/**/css/*','web/webroot/_ui/responsive/**/js/*','web/webroot/_
ui/responsive/**/images/*'],
       },
    },
    //...
});
```

For the live reload feature to work, the livereload.js script, served under the configured URL, needs to be included in page template:

Calling the automatic reload script in page template

```
<script src="//localhost:35729/livereload.js"></script>
```

The live reload feature should be enabled only in development environment and never included in production code, as it considerably impedes performance. To avoid accidental

enabling in production, the aforementioned script can be called from the <storefrontextension>/web/webroot/WEB-INF/tags/shared/debug/debugFooter.tag, which is rendered only when the storefront.show.debug.info property is set to true - this should never occur in production environment.

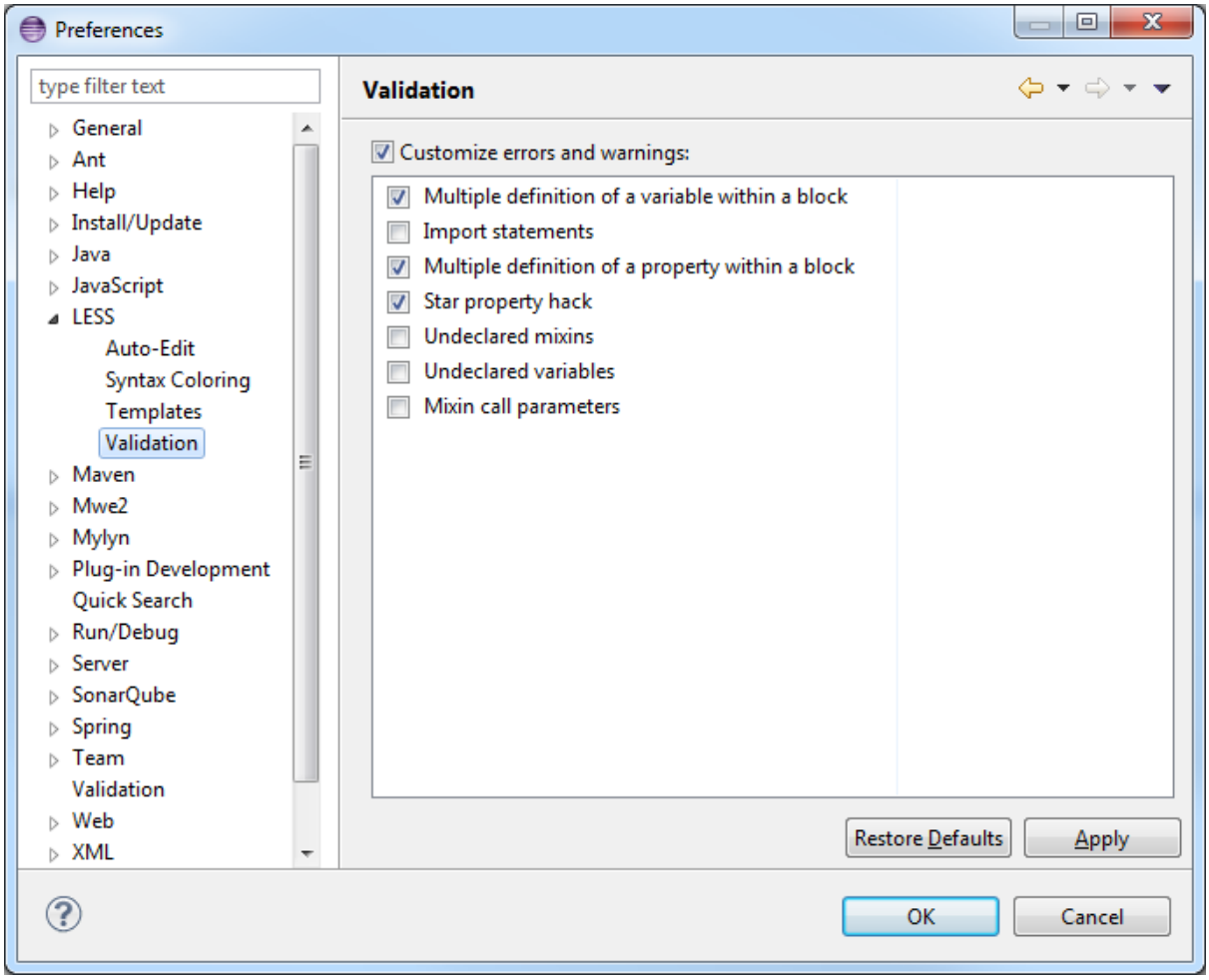
3 LESS and CSS Tools and Utilities

3.1 IDE Support

3.1.1 Eclipse

There is an Eclipse plugin available for LESS syntax highlighting and formatting: http://marketplace.eclipse.org/content/eclipse-plugin-less.

When using the Eclipse plugin, it may be necessary to disable certain validation options to avoid false alerts:



If errors occur during plugin installation, it may be necessary to install XText first.

3.1.2 IntelliJ

Current versions of IntelliJ offer native support for LESS.

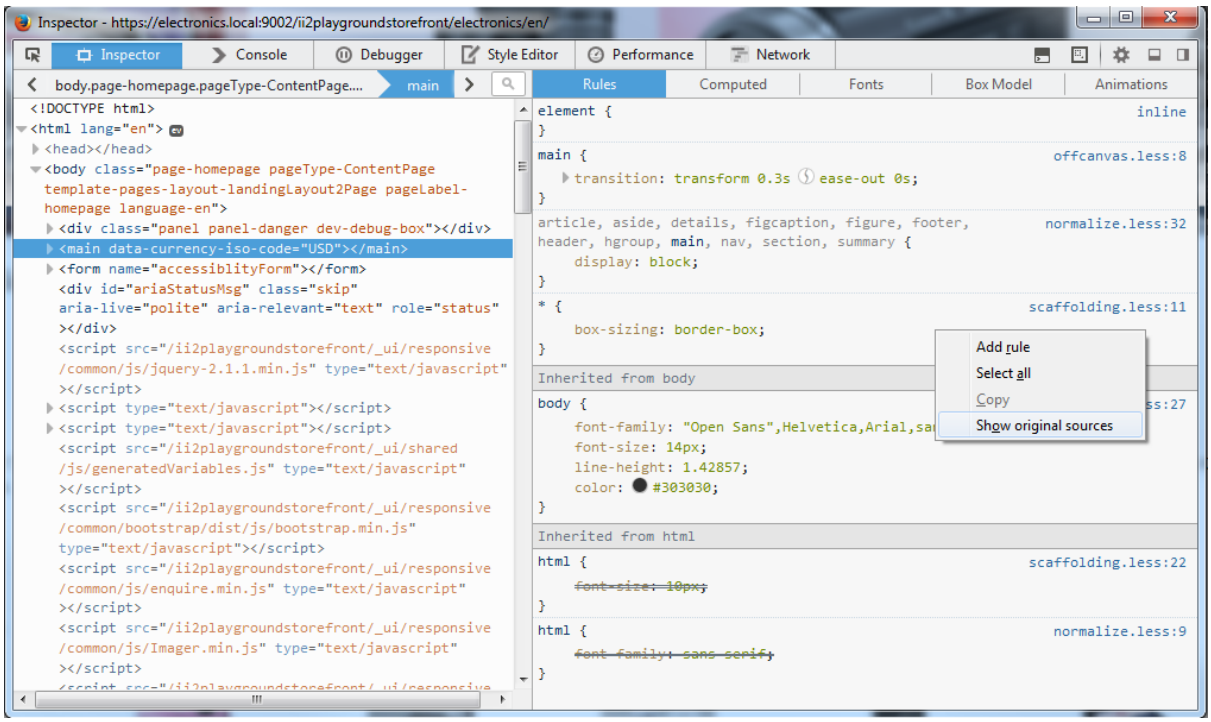
3.2 Browser Tools

To be able to see original LESS sources in browser's developer tools, LESS stylesheets need to be compiled with a source map option. This facilitates the development but has a huge impact on both the network and the browser performance. Therefore stylesheets **must** be recompiled without source maps for production and if tracking final CSS files before committing the work.

Trying to open CSS with source map may cause some problem in Eclipse, but these files should not be edited directly anyway.

3.2.1 Firefox

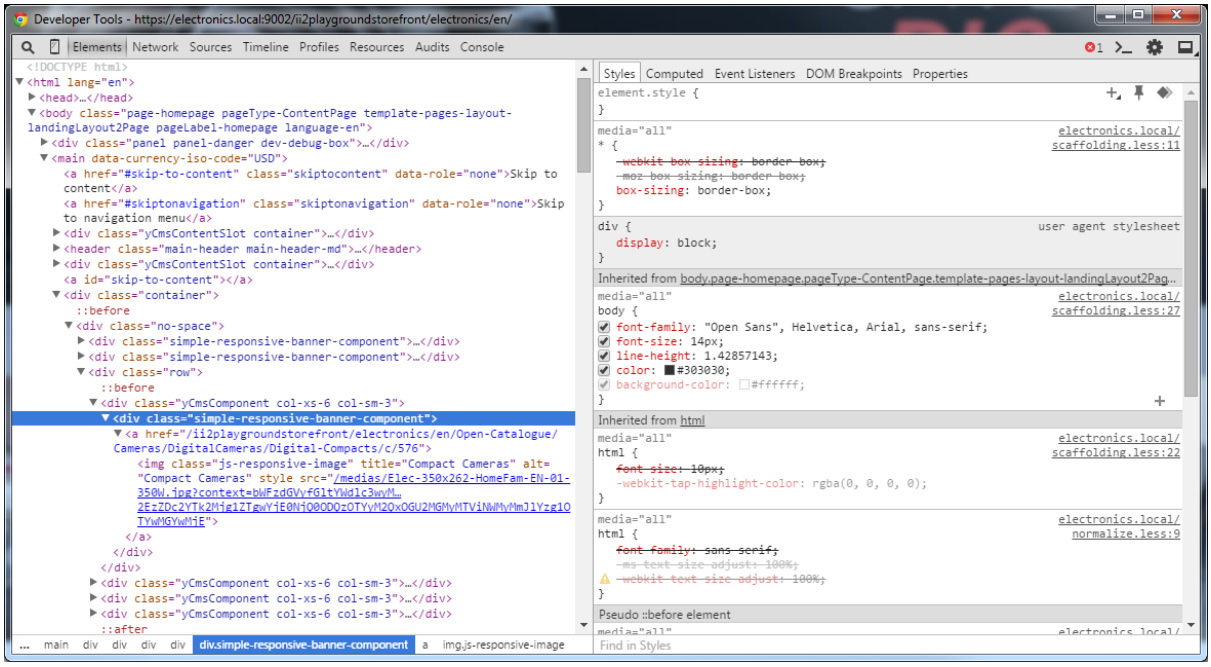
In Firefox, it is possible to see and edit LESS sources in browser's native *Inspect Element* panel. To do so, open the panel, right-click in *Rules* area and select the *Show original sources* option.



Unfortunately, at the time of writing this document there was no reliable plugin for Firebug.

3.2.2 Chrome

In Chrome, if the source map info is present in CSS file, the original LESS sources will be automatically displayed in the *Inspect Element* panel.



4 Additional Development Tools

To further streamline development work, additional Grunt tools can be included. The ones listed below proved to be useful during several implementation projects.

4.1 Sample Configuration

Sample configuration files have been attached to this page (please note that this is very basic configuration and will need to be adjusted to individual project needs). This configuration can be either merged into the standard project build or used as entirely separate utilities set.

These tools are meant as shortcuts for day-to-day developer's work. They are **not** meant to be used in any way in production environment nor are they intended for external distribution. They have not been tested with big batches of input data nor checked against corner cases and may not be bullet proof.

All listed tools are available under the MIT license, however, they are **not** a part of a standard hybris platform setup and should be evaluated before using them in any project.

4.1.1 How to Use

You need to have Node.js, Grunt and Grunt CLI installed first.

- Unzip the css-tools.zip into the destination folder.
- Inside the css-tools folder, open console and install project's dependencies:

```
> npm install
```

- Verify the project configuration in Gruntfile.js and adjust if necessary
- The sample configuration defines the storefront path as yacceleratorstorefront extension in hybris installation located next to the css-tools directory. This will most

likely need to be changed. Also, settings of individual tasks within the config folder may need some adjustments.
- Run desired tasks according to the description below.

Provided tasks can also be integrated into standard Grunt build process in storefront extension's Gruntfile.js; this, however, requires careful evaluation due to potential compatibility, security and licensing issues.

4.2 Additional Grunt Tasks for Build Process

4.2.1 Using project properties in Grunt build process

Sometimes it may be convenient to be able to use project properties within the Grunt build configuration. This allows for example enabling developer mode so the LESS files are compiled with source maps, live reload is active etc. or importing the generated variables to be used in Grunt build process. This can be achieved by translating specified .properties files into JSON and importing it into Gruntfile.js. The grunt-properties-to-json task enables specifying source files, filtering keys by the regular expression and saving retrieved properties as a JSON file.

Note that the task retrieves property values as defined in specified files, **not** the actual values of those properties in the environment. Getting the actual values may require reconfiguring the task to include all relevant *.properties files in proper order.

While grunt-properties-to-json provides an option for creating nested JSON structure by splitting keys on specified separator, it proved to cause issues when applying to standard Accelerator project.properties files, so it should not be used. Using flat JSON indexed by full property keys is recommended instead.

Task	propertiesToJSON
Input	project.properties in responsive storefront extension's folder the path is based on Grunt's configuration storefrontPath variable; task configuration in config/propertiesToJSON.js file may be adjusted to specify list of files to include or regular expression to filter property keys
Output	properties.js file in _results/properties-to-json folder
Usage example	grunt propertiesToJSON retrieves all properties form storefront's project.properties and saves them as JSON object in _results/properties-to-json/properties.js

4.2.2 LESS/CSS Postprocessing

The PostCSS tool enables performing additional transformations (formatting, compressing, linting etc.) to the CSS code. One of the most popular applications is adding required vendor prefixes to style properties using the Autoprefixer plugin.

As of Bootstrap version 3.2.0 using mixins defined in vendor-prefixes.less is deprecated and it is not optimal anyway, as they contain a predefined set of prefixes which does not take into

account project-specific browser requirements or current support for unprefixed properties. Autoprefixer offers more flexibility, allowing to generate only prefixes necessary for specific browser versions. Unfortunately, in most projects, it cannot be applied to the compiled CSS file due to lack of the equivalent Java implementation, which makes it impossible to configure the Ant build to achieve the same result. The solution is to execute Autoprefixer on custom modules-*.less file, when the necessary mixins are defined. It can be done once, at the project setup (unless the browser requirements change during implementation) and the task settings can be either configured directly in Gruntfile.js or read from project properties.

There seem to be some issues with prefixing LESS files even if the LESS syntax is declared in the task. Due to this issue, it is recommended to create separate file defining vendor prefix mixins, adhering to the CSS syntax, and process this file only. This allows using proper LESS anywhere else while the vendor prefix mixins can still be autoprefixed.

Task	postcss		
Available targets	**name**	**purpose**	
	default	processing of compiled style.css files	
	lessVendors	processing of vendor prefix LESS mixins (recommended usage)	
Input	style.css in responsive theme's css folder for the default target modules/modules-vendors.less in responsive theme's less source folder for the lessVendors target theme path is based on Grunt's configuration themeStylesPath variable		
Output	source files are modified		
Options	**name**	**values**	**notes**
	theme	theme name (without theme-prefix), e.g. alpha	if not present, the audit will be run for all responsive themes
	browsers	browser list as per Browserslist specification, e.g. "Firefox > 3.2, > 5%"	defaults to "> 1%, last 2 versions, Firefox ESR" (current Browserslist default)
Usage example	grunt postcss:lessModules --browsers="Firefox > 3.2, > 5%" recommended usage; adds vendor prefixes appropriate for specified browsers to the modules/modules-vendors.less in all responsive themes grunt postcss:default --theme=alpha adds vendor prefixes for the default browser configuration to style.css file of *alpha* theme		

4.3 Helpers and Utilities

4.3.1 Code Quality Reports

CSS Lint

The grunt-contrib-csslint task enables performing a CSS Lint audit on theme's stylesheet to identify syntax errors and potential performance issues. Custom settings can be specified for warning and error level of violation reporting. The report is generated as XML file in specified location.

The CSS Lint levels in sample configuration have been set to:

- ignore adjoining classes, box model, compatible vendor prefixes, duplicate background images, gradients, floats and qualified headings rules (either to properly style OOTB Accelerator templates or to avoid unnecessary code intended for outdated, not supported browsers),
- throw error at star and underscore hacks,
- issue a warning for other rules.

If error-level rules are broken, the task will fail with an *Aborted due to warnings* message. When evaluating more than one file, use the -f flag to force proceeding with subsequent files.

Task	csslint		
Input	style.css in responsive theme's css folder theme path is based on Grunt's configuration themeStylesPath variable		
Output	console XML file in _results/css-lint folder		
Options	**name**	**values**	**notes**
	theme	theme name (without theme-prefix), e.g. alpha	if not present, the audit will be run for all responsive themes
Usage example	grunt csslint runs CSSLint for style.css files in all responsive themes; saves audit results in _results/css-lint/css-lint/csslintComments-all.xml grunt csslint --theme=alpha runs CSSLint for style.css file of *alpha* theme; saves audit results in _results/css-lint/css-lint/csslintComments-alpha.xml		

CSS Count

The grunt-css-count task counts CSS selectors and rules and calculates nesting depth. This allows to avoid overqualified, unnecessarily specific selectors.

The sample configuration is set to throw an error when:

- file contains more than 4095 selectors (limit for older IE versions),
- there are selectors deeper than 5 nesting levels.

Audit results are displayed in Grunt console.

If error-level rules are broken, the task will fail with an *Aborted due to warnings* message. When evaluating more than one file, use the -f flag to force proceeding with subsequent files.

Task	csscount		
Input	style.css in responsive theme's css folder theme path is based on Grunt's configuration themeStylesPath variable		
Output	console		
	name	**values**	**notes**
Options	theme	theme name (without theme-prefix), e.g. alpha	if not present, the audit will be run for all responsive themes
Usage example	grunt csscount runs CSS Count for style.css files in all responsive themes grunt csscount --theme=alpha runs CSS Count for style.css file of *alpha* theme		

Colorguard

The grunt-colorguard task evaluates similarity of colors used in stylesheet, allowing to eliminate almost-but-not-quite-the-same colors. The similarity threshold can be set to custom value (in 0-100 scale - the lower, the more restrictive).

Audit results are displayed in Grunt console.

Task	colorguard		
Input	style.css in responsive theme's css folder theme path is based on Grunt's configuration themeStylesPath variable		
Output	console		
	name	**values**	**notes**
Options	theme	theme name (without theme-prefix), e.g. alpha	if not present, the audit will be run for all responsive themes
Usage example	grunt colorguard runs Colorguard for style.css files in all responsive theme grunt colorguard --theme=alpha runs Colorguard for style.css file of *alpha* theme		

4.3.2 Image Minification

These task enable stripping unnecessary information and compressing images to reduce file size.

Resulting files should be always evaluated for quality loss before being used.

Pngmin

The grunt-pngmin task operates on PNG files in location specified as source, performs minification with configured settings and saves resulting files in location specified as destination.

Task	pngmin		
Input	PNG images in _sources/pngmin directory		
Output	PNG images in _results/pngmin directory (if the --force flag is used, any existing files will be overwritten; otherwise they'll be skipped)		
Options	**name**	**values**	**notes**
	ext	output file extension, e.g. -minified.png	defaults to .png
	quality	output file quality range in "min-max" format with min and max values between 0 and 1000, e.g. "30-50"	defaults to "0-100", i.e. no restrictions
Usage example	grunt pngmin --ext=".PNG" --quality="30-50" processes all *.PNG files in _sources/pngmin directory with 30-50 quality settings and saves minified files in _results/pngmin directory		

Imagemin

The grunt-imagemin task works similarly to Pngmin, but can minify JPG. GIF, PNG and SVG files.

Task	imagemin		
Input	Supported types of images in _sources/imagemin directory		
Output	Images in _results/imagemin directory (any existing files will be overwritten)		
Options	**name**	**values**	**notes**
	png-optimization	Resulting PNG quality, between 0 and 7	defaults to 3 the higher value, the smaller (potentially) file and slower execution

			For some reason the option seems to affect also JPG compression.
	jpg-progressive	Progressive JPG compression, true or false	defaults to true
	gif-interlaced	Interlaced GIF conversion, true or false	defaults to true
Usage example	grunt imagemin --png-optimization=4 --jpg-progressive=false --gif-interlaced=false processes all *.png, *.jpg, *.gif and *.svg files in _sources/imagemin directory according to specified settings and saves the minified files in _results/imagemin directory		

Base64 conversion

For some resources, e.g. small images, it may make sense to embed them into CSS directly instead of performing an additional request. The grunt-data-uri task converts specified images to Base64 format and prepares index of converted resource.

The grunt-data-uri task operates on stylesheets referencing resources to be converted. For developer's convenience, there is a wrapper imageToBase64 task prepared that creates index stylesheet of all images in the _sources/datauri directory and runs grunt-data-uri on this stylesheet, so the manual stylesheet creation is not necessary.

The CSS output shape is controlled by template.css file in _sources/datauri directory. Be careful when modifying this file, as the original *DataUri* task is rather greedy when it comes to finding resources - it will convert comments, CSS class names etc., resulting in virtually illegible output file.

Task	imageToBase64		
Input	Images in _sources/datauri directory		
Output	images.css file in _results/datauri directory (any existing files will be overwritten)		
	name	**values**	**notes**
Options	max-size	Maximum sixe of resources that will be converted, in bytes	defaults to 2048 images exceeding that size will not be converted, but simply copied to output file
Usage example	grunt imageToBase64 --max-size=4096 prepares CSS index of all images in _sources/datauri directory, converts those		

smaller than 4KB into Base64 and saves the resulting images.css file in _results/datauri directory

4.3.3 Webfont Generation

The grunt-webfont task converts set of SVG images to a webfont, enabling usage as an icon font (similarly to Bootstrap Glyphicons or FontAwesome) for scalable images that can have text effects (colors, shadows etc.) applied to them. Generates sample CSS and HTML files for an usage example.

By default, the grunt-webfont task processes all the SVG files in source directory; this may result in reordering webfont glyphs if modifying previously created font. It is recommended to force proper glyph ordering by naming SVG files in a way that will ensure consistent sorting (e.g. prefixing image name with ascending numbers).

In some cases, changes in task options do not cause font regeneration (a *Font <name> wasn't changed since last run* message is displayed). Remove output files from the results directory to force regenerating.

TaskwebfontInputSVG images in _sources/webfont directoryOutput

Font, HTML and CSS files in _results/webfont directory

Optionsnamevaluesnotesfont-name

Name of font, e.g. "my-font"

defaults to "y-icons"

will be used also for file name and CSS class prefix

Usage example

grunt webfont --font-name="my-icon-font"
creates my-icon-font webfont with glyphs based on SVG images in _sources/webfont directory; saves webfont files and sample HTML and CSS in _results/webfont directory

5 Usage Guidelines

The guidelines below are the basic style guide and best practices proposed for implementation projects. There is no guarantee nor expectation that they are observed in any OOTB platform code. Some adjustments and customizations for individual projects may be necessary.

Adhering to the defined style guide, like the one described below, helps to ensure:

- consistent, reusable code,
- minimized redundancy and risk of accidental overrides,
- improved network and browser performance,
- easier code maintenance.

Before you Start

- Take a glance at the LessCSS documentation.
- Get acquainted with what Bootstrap has to offer.

5.1 General Rules

Try to reuse as much of existing code as possible

In this particular case, *possible* should be understood as *technically doable and makes sense*. There is no point in using an existing mixin or extending previously defined class only to override half of its styles in the next line.

- Check the Bootstrap library and ybase components for existing solutions
- Take advantage of various *-variant mixins defined in Bootstrap to define new flavours of existing components (e.g. buttons, alerts, panels)
- There are two ways of using an existing class: you can add it to an HTML element in your JSP/tag file or mix it in in the LESS stylesheet. Consider which one fits your needs better.

5.1.1 Write DRY Code

Don't repeat yourself.

- Create variables
  If you introduce a value that is used at least twice, store it in a variable.
- Define mixins
  If you have written the same few rules more than once, it's probably time to create a mixin. Maybe even a parametrized one.

5.1.2 Keep the Specificity Low

Use the simplest, shortest selector that does the job. Avoid overqualified selectors.

5.2 Project Structure

5.2.1 Modules

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/modules.less

This file contains universal, project-agnostic modules - e.g. utility mixins for element positioning. Vendor prefix mixins also live here.

If using Autoprefixer, it may be necessary to separate the code that needs to be prefixed (e.g. by creating the modules/modules-vendors.less file) due to the LESS syntax issues in PostCSS.

- Define abstract building blocks in this file.
- no dependencies (in other words, do not use Bootstrap/ybase/theme-specific variables or mixins);
- non-theme specific default values for mixin parameters, when applicable;

- only widely reusable elements.
- This is not a place to define any particular look and feel for elements.

This file is meant to be shared between various themes and projects. The mixins defined here are, for the most part, not to be used directly, but rather mapped to or used by theme-specific elements in mixins.less.

If needed, it can be divided further into modules-*.less files. In such case, they should be imported into modules.less and no modules definitions should be left directly in components.less.

⚠No direct styling of any elements in this file.

5.2.2 Theme Variables

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/custom-variables.less

This file contains all theme-specific variables and overrides for Bootstrap, ybase and original theme variables. If it is more convenient, these variables van be defined or overwritten directly in the theme-variables.less file, although having a custom, theme-specific file makes it easier to evaluate the scope of changes (e.g. when reusing theme in subsequent projects).

- Define all global theme-specific variables at the beginning of this file.
  This means no global variables in imported files. Store them all in one place to avoid unmaintainable, untrackable mess.
- If needed, map them to Bootstrap and ybase variables below.
  It's not strictly required to define theme-specific variable for the sole purpose of overriding a Bootstrap global, but if it improves readability (e.g. to indicate that @link-color is set to top banner background color), go for it.

⚠No direct styling of any elements in this file.

5.2.3 Theme Mixins

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/mixins.less

This file contains mapping of modules.less to a theme-specific look and feel, as well as any other theme-specific mixins necessary.

- Create theme-specific instances of modules, using variables from variables.less
- Define any mixins needed for current theme that are not likely to be reused in other projects.

⚠No direct styling of any elements in this file.

5.2.4 Theme Components

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/components.less

This file contains styling of theme-specific components that are reused on different pages across the site, e.g. form elements, messages, tabs, shopping cart block etc.

To avoid exceedingly big files, if needed, it can be divided further into components-*.less files, based on component types or groups (e.g. components-lists.less or components-

toolbars.less). In such case, they should be imported into components.less and no direct styling should be left in components.less.

5.2.5 Theme Elements

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/elements.less

This file contains styling of one-off elements that are not repeatedly used, e.g. formatting of a specific block that occurs only on a single page.

To avoid exceedingly big files, if needed, it can be divided further into elements-*.less files, based on pages or site sections they are used in (e.g. elements-checkout.less or elements-account.less). In such case, they should be imported into elements.less and no direct styling should be left in elements.less.

5.2.6 Temporary Adjustments

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/todo.css

Sometimes it is necessary to cooperate with external teams that do not have LESS competencies. If training is not feasible and volume of code they deliver is relatively small, it may be convenient to have the external team deliver pure CSS code which is then backported to proper LESS structure by the internal team. This prevents polluting LESS sources with hardcoded variable values, non-optimized code etc. In this case, the code should be placed in todo.css file and translated into proper LESS code as soon as possible. This should be treated as the last resort solution - onboarding the external teams into LESS process is always preferable.

⚠This file should be removed before going to production.

5.2.7 Main theme stylesheet

web/webroot/WEB-INF/_ui-src/responsive/themes/<theme_name>/less/style.less

This file imports (directly or indirectly) all theme files and is compiled to main theme stylesheet.

⚠No direct styling of any elements in this file.

5.3 Naming Conventions

- Use kebab-case notation whenever possible.
- Prefix names of custom variables and mixins with a fixed prefix (for sample setup it's y-*) to distinguish between theme-specific and inherited elements.
- Use descriptive names. Name variables and mixins for their role, not their current content.

```
@top-banner-padding: (@padding-base-vertical / 2); // NOT @half-padding
@footnote-font-size: .75em; // NOT @smaller-font
```

5.4 Code Formatting

- Indent code using TAB characters, starting at 0 column; in IDE set TAB width to 4 characters.
- For block of code, place the opening bracket in the same line as block name/selector and the closing bracket in the separate line. Put a single space between the name and opening bracket.

```
.block-name {
    // rules
}
```

- Leave single blank line between top-level blocks or between end of one block and a comment describing the subsequent one.

```
.block-one {
    // rules
}

// Comment describing .block-two
.block-two {
    // rules
}
```

- Do not leave a blank line between a comment and a block it describes (see above).
- For selector groups, place every selector in a separate line.

```
tag .containing-class,
.other-class,
.class-two:pseudoclass {
    // rules
}
```

- Within the code blocks, put rules in separate lines, always ending with a semicolon ;.
- Indent the block's content by one tab.
- Place the block's content into groups in following order:
- local variables,
- extending classes, ids and mixins,
- element-specific styles, subsequently:
- content (for pseudoelements),
- position-related properties (position, coordinates),
- z-index,
- display,
- box-model related properties (box-sizing, width, height, margins, padding, overflow),
- borders and outlines,
- typography-related properties (font, line-height, word-wrap, text-decoration, text-overflow),
- colors and backgrounds;

- pseudoelement rules,
- nested elements,
- pseudoclass rules.
- Leave single blank line between groups; do not leave blank lines before the first group and after the last one.

```
.my-selector {
    @variable: "value";

    .include-mixin();

    position: relative;
    width: 100%;
    font: normal 100%/1.5em sans-serif;

    &:before {
        content: @variable;
        position: absolute;
        left: 0;
        top: 0;
    }

    .containing-class {
        font-weight: bold;
    }

    &:hover,
    &:focus {
        outline: dotted 1px red;

        .containing-class {
            text-decoration: underline;
        }
    }
}
```

5.5 Nesting Selectors

LESS enables nesting selectors, which allows writing more legible, concise code, but at the same time poses a risk of creating unnecessarily specific rules.

- Always use the simplest possible selector.
- Use nested selector only when they are strictly related functionally, not because one accidentally appears within the other in a particular HTML document.
- Use only meaningful selectors, do not list the whole DOM in between.
- As a rule, do not nest more than 3 levels.
  If an exception is really necessary, explain in comments.
- Pay attention to further depth introduced by included mixins.
  If you include a mixin into your selector, any nested hierarchy in a mixin will be

effectively included in the containing selector. This can lead to very high specificity and heavily overqualified selectors, which hurts performance.

5.6 Data Types

5.6.1 Numbers and Units

- Do not use units for zero value.
- Do not write trailing zeros; for decimal values, less than 1 do not write leading zeros.

```
p {
    font-size: .75em;
    margin: 0;
}
```

- Use proper math, not unit concatenation. Wrap math operations in parentheses.

```
p {
    line-height: (2 * @line-height-base * 1em);
}
```

- Do not write magic numbers. When calculating value based on other numbers, leave the math in the code or, if it's very complex, at least in the comments.
- Consider defining a variable for reused values (the ones with the same meaning, obviously - it doesn't make much sense to use the same variable for block margin and color darkening factor just because they both accidentally equal to 20%).

5.6.2 URLs

- Wrap URLs in single quotes.

```
p {
    background: url('images/my-image.png');
}
```

5.6.3 Colors

- Do not use colors directly in styling. Define color variables in variables.less.
- When calculating color value based on other color, write is as an operation, not a resulting color.

```
@top-banner-background: #bada55;
@top-banner-lettering: darken(@top-banner-background, 35%);
```

- Write colors as hex codes. Shorten the notation to three-digit hex whenever possible. For alpha transparency, use either fadeout function or rgba() notation.

```
@footer-banner-color: #bad;
@footer-banner-background: fadeout(@footer-banner-color, 70%);
```

5.7 Mixins

- Always use parentheses when mixing in other selectors and mixins.

```
.my-selector {
    .other-selector();
    .include-mixin();
}
```

- Write parentheses when defining mixin that is intended to be used as a mixin only (in other words, not for direct use as a class or id)

```
.my-mixin() {
    // rules
}
```

- Separate mixin arguments with a semicolon.

```
.my-mixin(@arg1; @arg2){
    // rules
}
```

- For guarded mixins, put common properties in non-guarded block; the guarded blocks should contain only properties specific to the condition.

```
.mixin(@color){
  background: @color;
  padding: 10em;
}
.mixin(@color) when (lightness(@color) > 50%) {
  color: darken(@color, 30% );
}
.mixin(@color) when (lightness(@color) <= 50%) {
  color: lighten(@color, 30% );
}
```

5.8 Media Queries

Standard Bootstrap responsive utilities (hiding elements, grid layouts etc.) depend on breakpoints defined by generated variables, shared between CSS, JS and JSP. These breakpoints should be modified **only** in resources/generateVariables.properties file, as doing otherwise can break the responsive behaviors.

- Define reusable, named media queries in variables.less
- Use variables for breakpoints
- Use descriptive names for media queries

```
// variables.less
@y-media-wide-landscape-screen: ~"screen and (min-width: @{screen-lg-min}) and
(orientation: landscape)";
```

```
// elements.less
@media @y-media-wide-landscape-screen {
   // rules
}
```

5.9 Vendor Prefixes

- Avoid manually adding vendor-prefixed properties in direct styles - use prefixing mixins and modules from modules.less instead.
  These mixins can be automatically updated by the means of PostCSS to current browser requirements, resulting in the least possible amount of non-standard code.
- Consider adding new prefixing mixins to modules.less if the existing ones are insufficient.

5.10 Comments

- Comment the code.
- Put descriptive comments at least at top-level code blocks.
- Place the comments:
- after the semicolon in the related line - for very short comments describing the meaning of single property;
- above the line - for longer comments describing one or more of subsequent lines;
- directly above the block - for comments documenting the block purpose.

```
// Description of the block purpose,
// contents, usage etc.
.my-block{
   @var: "value"; // variable meaning

   //this element will be used as a container
   //for its absolutely positioned descendants
   position: relative;
}
```

- Beware of LESS-style single-line comments in files that will be processed by PostCSS (at least modules.less) - to avoid compilation errors wrap them in block comments.

```
/*
// My comment
*/
```

6 References

6.1 Links

https://wiki.hybris.com/display/hybrisALF/Responsive+Coding+Standards#ResponsiveCodingStandards-UsageGuidelines